

Optimize4: An optimization package for *morpho*

June 27, 2026

Chapter 1

Overview

The `optimize4` package facilitates the solution of optimization problems, with a particular focus on shape optimization problems for which *morpho* was designed. The design separates *what* is being optimized (an *OptimizationProblem*), *how parameters are exposed* to an algorithm (an *OptimizationAdapter*), and *how the algorithm proceeds* (an *OptimizationController*). To use the package, import it as usual:

```
import optimize4
```

This imports three subsidiary modules that work together (Fig.).

OptimizationProblem describes energies and constraints using morpho Functionals.

OptimizationAdapter reads and writes degrees of freedom, and evaluates the objective, constraints, and derivatives. Adapters can be chained (penalty methods, fixing variables, caching, and so on).

OptimizationController implements an optimization algorithm by calling adapter methods and updating parameters.

A typical mesh problem:

```
var problem = OptimizationProblem(mesh)
problem.addEnergy(Length())
problem.addConstraint(AreaEnclosed())
var adapter = ProblemAdapter(problem, mesh)
```



Figure 1.0.1: **Classes in the `optimize4` package** and how they interact.

```
var opt = SQPController(adapter)
opt.optimize(500)
```

A field problem optimizes degrees of freedom stored in a `Field` rather than mesh vertex positions:

```
var problem = OptimizationProblem(mesh)
problem.addEnergy(GradSq(phi))
var adapter = ProblemAdapter(problem, phi)
var opt = LBFGSController(adapter)
opt.optimize(500)
```

1.1 Choosing a controller

The following pairings are typical guidelines for selecting an `OptimizationController`. Adapter composition (for example `PenaltyAdapter` or `FixAdapter`) and problem scale often matter as much as the constraint structure.

Problem type	Suggested controller
Unconstrained mesh or field problem	LBFGSController
Constrained problem with a small number of equality constraints	SQPController
Constrained problems, particularly with inequality constraints	PenaltyController
Optimization of regular functions	FunctionAdapter + LBFGSController or SQPController

Chapter 2

Mathematical formulation

We seek

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & \mathbf{c}(\mathbf{x}) = 0 \\ & \mathbf{d}(\mathbf{x}) \leq 0 \end{aligned} \tag{2.0.1}$$

where $f(\mathbf{x})$ is the objective, $\mathbf{x} \in \mathbb{R}^N$ are parameters, $\mathbf{c}$ collects $M_{=}$ equality constraints, and $\mathbf{d}$ collects $M_{\neq}$ inequality constraints. In `optimize4`, inequalities added with `onesided=true` follow the convention $d_i(\mathbf{x}) \leq 0$ in the feasible region; positive values indicate violation or inactive local constraints depending on context.

We write $\mathbf{g} = \nabla f$, $\mathbf{H} = \nabla^2 f$, and constraint Jacobians $\mathbf{C}, \mathbf{D}$ with entries $C_{ij} = \partial c_j / \partial x_i$. Subscripts k denote iteration indices.

Chapter 3

OptimizationProblem

An *OptimizationProblem* is a container for morpho Functionals used as energies (objective terms) or constraints. It holds the mesh, lists of energies, global constraints, and local constraints.

3.1 Energies

Add terms to the objective with `addEnergy`:

```
var en = problem.addEnergy(functional)
```

The returned `Energy` object may be modified using optional parameters `selection`, `prefactor`. The total objective is the sum of all energy terms, each evaluated via `functional.total(mesh)` (and `functional.gradient` when required).

3.2 Constraints

Global constraints restrict integrals of functionals:

```
var cons = problem.addConstraint(functional, selection=nil, field=nil)
```

The target value is fixed at the integral of the functional when the constraint is added. Local constraints apply at mesh elements (for example a level-set constraint at each vertex):

```
var cons = problem.addLocalConstraint(functional, selection=nil, field=nil,  
onesided=false, target=0)
```

Set `onesided=true` for inequalities $c \leq 0$. Set `field=` when a functional depends on a particular `Field` (needed when updating after refinement).

3.3 Example

Minimize perimeter at fixed enclosed area:

```
var problem = OptimizationProblem(mesh)  
problem.addEnergy(Length())  
problem.addConstraint(AreaEnclosed())
```

3.4 Updating after refinement

After `MeshRefiner` or similar, call `problem.update(refmap)` with a dictionary mapping old mesh, fields, and selections to new ones:

```
var refmap = ref.refine(selection=srefine)
problem.update(refmap)
mesh = refmap[mesh]
field = refmap[field]
```

Chapter 4

OptimizationAdapters

Optimization algorithms should not depend on whether parameters live on a mesh, in a field, or in a plain vector. An *OptimizationAdapter* exposes a uniform interface: set parameters, evaluate f , $\mathbf{g}$, constraints, and constraint derivatives. Adapters may be composed: for example a `PenaltyAdapter` wrapping a `ProblemAdapter` converts a constrained problem into an unconstrained one for `LBFGSController`.

4.1 The adapter interface

An *OptimizationAdapter* maps parameters $\mathbf{x} \in \mathbb{R}^N$ to the objective $f(\mathbf{x})$, gradient $\mathbf{g} = \nabla f$, and constraint values. Equality constraints $\mathbf{c}(\mathbf{x}) = \mathbf{0}$ and inequalities $\mathbf{d}(\mathbf{x}) \leq \mathbf{0}$ (including local constraints sampled on mesh elements) are returned as lists of scalars or matrices; Jacobians appear as lists of column matrices $\partial c_j / \partial \mathbf{x}$ stacked column-wise when a constraint has multiple components.

Every adapter implements at minimum:

set($\mathbf{x}$) / **get()** — parameters as a column `Matrix`.

value() / **gradient()** — objective and $\mathbf{g}$.

directionalDerivative($\mathbf{d}$) — $\mathbf{g}^\top \mathbf{d}$ by default; required when the merit function is nondifferentiable (`L1PenaltyAdapter`).

countConstraints() / **countEqualityConstraints()** / **countInequalityConstraints()**

constraintValue() / **constraintGradient()** — values and Jacobian columns. Inactive one-sided inequalities ($u > 0$) are zeroed in **constraintValue()**.

constraintValueForActiveSet() — raw inequality values for active-set logic (feasible $u > 0$ retained).

hessian() / **constraintHessian()** — optional second derivatives.

Adapters compose by wrapping: a `PenaltyAdapter` outside a `ProblemAdapter` converts a constrained morpho problem into an unconstrained objective for `LBFGSController`; `FixAdapter` inside the chain enforces boundary conditions.

4.2 Adapters

4.2 FunctionAdapter

The simplest adapter wraps callable morpho functions. It is the natural entry point for understanding the interface before moving to mesh and field problems. Consider minimizing $(x - \frac{1}{2})^2 + (y - 1)^2 + \frac{1}{4}xy$:

```

fn func(x, y) {
    return (x-0.5)^2 + (y-1)^2 + 0.25*x*y
}
var adapt = FunctionAdapter(func, start=Matrix([0,0]))

```

Optional gradient and hessian callables may be supplied. When omitted, centered finite-difference approximations are built from shared kernels (the same routines used by `FiniteDifferenceAdapter`). For coordinate i with adaptive step h_i ,

$$\frac{\partial f}{\partial x_i} \approx \frac{f(\mathbf{x} + h_i \mathbf{e}_i) - f(\mathbf{x} - h_i \mathbf{e}_i)}{2h_i}, \quad (4.2.1)$$

with $h_i = \max(\epsilon_{fd}, \epsilon_{fd}|x_i|)$ and $\epsilon_{fd} = (10^{-16})^{1/3}$ (similar scaling for the Hessian with $\epsilon_{hess} = (10^{-16})^{1/4}$).

Constraints are lists of callables with optional `constraintgradients`, `constraint Hessians`, and `equalitycount` (how many leading constraints are equalities; the remainder are inequalities). Per-constraint derivatives fall back to the same FD machinery when not supplied.

`FunctionAdapter` is ideal for prototyping, teaching, and regression tests. For morpho shape problems built from `Functionals`, use `ProblemAdapter` instead.

4.2 ProblemAdapter

`ProblemAdapter` connects an `OptimizationProblem` to the adapter interface. Pass one or more optimization targets (Mesh or Field objects):

```

var adapter = ProblemAdapter(problem, mesh)
var adapter = ProblemAdapter(problem, field)
var adapter = ProblemAdapter(problem, mesh, field)
var adapter = ProblemAdapter(problem, f1, f2)

```

Parameter vector. With a single target, `get()` and `set(x)` use that object's degrees of freedom (columnized vertex positions for a mesh; `Field.linearize()` for a field). With multiple targets, degrees of freedom are concatenated in target order so that

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_{\text{mesh}} \\ \mathbf{x}_{\text{field}} \end{bmatrix}. \quad (4.2.2)$$

Objective. The adapter sums all energies for `value()`:

$$f(\mathbf{x}) = \sum_k E_k(\mathbf{x}). \quad (4.2.3)$$

For each target it accumulates gradients from `functional.gradient(mesh)` (mesh targets) or `functional.fieldgradient(mesh)` (field targets). Field gradients are included only when the functional's `field` property matches the target. Custom functionals must implement `fieldgradient` and declare their field dependency.

Constraints. Global constraints fix integrals of functionals at the value obtained when the constraint is added; local constraints apply at mesh elements (for example a level-set constraint at each vertex). The adapter evaluates `functional.total(mesh)` and `functional.integrand(mesh)`, splitting equality and one-sided inequality contributions. When `functional.grade` is absent, the mesh grade of integrands is inferred from the integrand width.

For active-set methods the method `constraintValueForActiveSet()` returns raw local inequality values. An inequality with value u is *active* when $u \leq 0$ (violated or on the boundary); interior feasible points with $u > 0$ are inactive. This matches the rule used by `LagrangeMultiplierAdapter` and `SQPController`.

Field indexing and boundary conditions. Use `selectionToIndexList(selection, target)` to map a `Selection` to linear indices in the parameter vector, then pass the list to `FixAdapter`. Works for mesh and field targets; with multiple targets indices are offset automatically. For vector-valued fields use `components=` to restrict which components are fixed:

```
var fix = adapter.selectionToIndexList(bnd, field)var fadapt =  
    FixAdapter(adapter, fix)
```

`ProblemAdapter` does not supply a Hessian; use `LBFGSController` or `SQPController`.

4.2 DelegateAdapter

`DelegateAdapter` forwards every `OptimizationAdapter` method to a wrapped adapter. It is the base class for most transform adapters (`ProxyAdapter`, `FixAdapter`, `DeconstrainAdapter`, `FiniteDifferenceAdapter`).

```
var wrapped = DelegateAdapter(baseAdapter)
```

Custom adapters can subclass `DelegateAdapter` and override only the methods they need to change, leaving all other calls transparently delegated.

4.2 DeconstrainAdapter

`DeconstrainAdapter` hides constraints from the adapter interface: `countConstraints()` returns zero and `constraintValue() / constraintGradient()` return `nil`, while `value()` and `gradient()` still evaluate the original objective on the underlying adapter. This is used inside `SQPController` when building an L-BFGS approximation to the *unconstrained* objective Hessian (`DeconstrainAdapter` wrapped around the primal adapter).

4.2 FixAdapter

`FixAdapter` holds selected coordinates of $\mathbf{x}$ at their current values. After `set(x)`, entries in a user-supplied index list are reset to the previous iterate; corresponding rows of ∇f and constraint Jacobians are zeroed, enforcing

$$\frac{\partial f}{\partial x_i} = 0, \quad \frac{\partial c_j}{\partial x_i} = 0 \quad (4.2.4)$$

for fixed indices i (the optimizer never moves those degrees of freedom). Hessians are not supported.

With `ProblemAdapter`, obtain indices from mesh or field selections rather than hand-counting vertices:

```
var fix = adapter.selectionToIndexList(selection, mesh)var fadapt =  
    FixAdapter(adapter, fix)
```

4.2 ProxyAdapter

`ProxyAdapter` caches evaluations of `value()`, `gradient()`, `hessian()`, and constraint methods at the current parameters. Calling `set(x)` clears the cache. Controllers automatically wrap adapters in a `ProxyAdapter` unless one is already present, so algorithm code can call `gradient()` multiple times per iteration without repeating expensive functional evaluations.

Diagnostic methods report how many evaluations actually reached the underlying adapter:

```
adapter.report()var count = adapter.countEvals()
```

The list returned by `countEvals()` is ordered [`value`, `gradient`, `hessian`, `constraintValue`, `constraintGradient`, `constraintHessian`].

4.2 DeflationAdapter

DeflationAdapter supports exploring multiple solutions of the same problem by adding inequality constraints that exclude neighborhoods of known minimizers. For each stored solution $\mathbf{x}_{\text{soln}}$ and deflation radius R , the adapter appends

$$\frac{\|\mathbf{x} - \mathbf{x}_{\text{soln}}\|_2}{R} - 1 > 0 \quad (4.2.5)$$

so that any point within distance R of a known solution is infeasible.

```
var dadapt = DeflationAdapter(adapter,
    radius=1)dadapt.addSolution(adapter.get())dadapt.jiggle(eps=0.1)
```

Duplicate solutions (within tolerance `eps`) are ignored. The method `jiggle(eps)` adds Gaussian noise to the current parameters to escape a basin before re-optimizing.

4.2 PenaltyAdapter

PenaltyAdapter converts a constrained problem into an unconstrained one by adding squared penalty terms. Let $\mathbf{v}(\mathbf{x})$ be the flattened constraint vector from `constraintValue()`, with inactive inequalities ($u > 0$) zeroed. The penalized objective is

$$Q_\mu(\mathbf{x}) = f(\mathbf{x}) + \mu \|\mathbf{v}(\mathbf{x})\|^2, \quad (4.2.6)$$

where μ is adjusted via `setPenalty(mu) / penalty()`.

When analytical Hessians are available on the underlying adapter,

$$\nabla Q = \nabla f + 2\mu \mathbf{C}^\top \mathbf{v}, \quad \nabla^2 Q = \nabla^2 f + 2\mu \sum_j \left(v_j \nabla^2 c_j + \nabla c_j \nabla c_j^\top \right), \quad (4.2.7)$$

where $\mathbf{C}$ stacks constraint gradient columns. Used with `PenaltyController`, μ is increased until $\|\mathbf{v}\|$ falls below a tolerance.

4.2 L1PenaltyAdapter

L1PenaltyAdapter defines a nondifferentiable merit function for constrained line searches:

$$\Phi_\mu(\mathbf{x}) = f(\mathbf{x}) + \mu \|\mathbf{v}(\mathbf{x})\|_1, \quad (4.2.8)$$

using the same flattened constraint vector $\mathbf{v}$ as `PenaltyAdapter`. Calls to `gradient()` and `hessian()` raise an error; controllers must use `directionalDerivative(d)`.

Along direction $\mathbf{d}$,

$$\Phi'_\mu(\mathbf{x}; \mathbf{d}) = \mathbf{g}^\top \mathbf{d} + \mu \sum_j \sigma_j \nabla c_j^\top \mathbf{d}, \quad (4.2.9)$$

where $\sigma_j = \text{sign}(v_j)$ when $v_j \neq 0$ and $\sigma_j = \text{sign}(\nabla c_j^\top \mathbf{d})$ when $v_j = 0$ (Clarke subgradient of the absolute value). Inactive inequalities are skipped. This merit function is used by `ProjectedGradientDescentController` and `SQPController` during line searches.

4.2 LagrangeMultiplierAdapter

LagrangeMultiplierAdapter augments the primal variables with Lagrange multipliers λ , forming the Lagrangian

$$\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda^\top \mathbf{c}(\mathbf{x}), \quad (4.2.10)$$

where inactive inequality multipliers are held at zero ($\lambda_i = 0$ when the corresponding $u > 0$). The combined parameter vector stacks $\mathbf{x}$ and λ ; `lagrangeMultipliers()` / `setLagrangeMultipliers(lambda)` access the multiplier block.

Gradients split into primal and dual parts:

$$\nabla_{\mathbf{x}} \mathcal{L} = \nabla f - \mathbf{C}^\top \lambda \quad (4.2.11)$$

$$\nabla_{\lambda} \mathcal{L} = -\mathbf{c} \quad (4.2.12)$$

(inactive inequalities zeroed in $\mathbf{c}$). The primal block is returned by `varGradient()`; the full KKT gradient stacks `varGradient()` and $-\mathbf{c}$. For the current active set, `activeConstraintSystem()` returns constraint values and Jacobian columns in one pass; `setActiveLagrangeMultipliers(lambda)` updates only multipliers on active constraints. The adapter is constructed internally by `SQPController(control.ladapter)` after `start()`.

4.2 ReprojectionAdapter

ReprojectionAdapter ignores the original objective and minimizes pure constraint violation

$$F(\mathbf{x}) = \|\mathbf{c}(\mathbf{x})\|^2 + \|\mathbf{d}^-(\mathbf{x})\|^2, \quad (4.2.13)$$

where $\mathbf{d}^-$ collects negative parts of inequalities (the same convention as penalty methods). $\nabla F = 2 \mathbf{C}^\top \mathbf{v}$ with $\mathbf{v}$ the flattened constraint vector. `ProjectedGradientDescentController` and `SQPController` minimize F with L-BFGS after primal steps when feasibility is lost.

4.2 FiniteDifferenceAdapter

FiniteDifferenceAdapter wraps any adapter that implements `value()` (and optionally constraints) but lacks derivatives, supplying `gradient()` and `hessian()` by centered differences of `value()` using Eq. 4.2.1 and the corresponding second-order stencil. Step sizes follow `_fdStepsize` so that perturbations are representable in floating point.

```
var fdadapt = FiniteDifferenceAdapter(adapter)
```

Each gradient entry requires two objective evaluations; a full Hessian requires $\mathcal{O}(N^2)$ evaluations, so this adapter is practical only for small parameter vectors (or when used indirectly through `FunctionAdapter` on low-dimensional test problems).

Chapter 5

OptimizationControllers

OptimizationControllers implement algorithms by calling adapter methods. For background on the methods below, see Nocedal and Wright, “Numerical Optimization” (Springer). New controllers can be added by subclassing and overriding `begin`, `step`, and `next`.

5.1 Reporting

OptimizationController objects support controllable levels of output. To set, initialize the OptimizationController with the optional parameter `verbosity` with one of the following options, supplied as a String

silent Suppresses all output except errors.

quiet Suppresses all output except errors and warnings.

normal Report progress of the controller, but suppress additional output

verbose Display all available information

5.2 Interface

An OptimizationController is a base class for iterative optimization algorithms. The user performs optimization by calling the `optimize` method with a maximum number of iterations.

Algorithm 5.1 Generic optimization algorithm

```
1: start()
2: record()
3: for  $i \leftarrow 1$  to  $N_{iter}$  do
4:   if hasConverged() then
5:     break
6:   end if
7:   iterate()
8:   report(i)
9:   record()
10: end for
```

The generic optimization sequence is shown in Algorithm 5.1. Particular algorithms are defined by subclassing `OptimizationController` and implementing the below methods, all of which do nothing in the base class unless otherwise indicated.

start() Called once by `optimize` to performs any initialization at the beginning of optimization.

begin() Calculate any necessary quantities at the beginning of each iteration.

step() Perform the optimization step

next() Calculate updated information at the end of the iteration.

iterate() Calls `begin`, `step` and `next` in turn.

record() Records information about the iteration. By default, the value of the objective functional is stored after each iteration step.

The `OptimizationController` base class provides two further methods that form part of the generic optimization algorithm:

hasConverged() Performs a convergence check as described in subsection 5.3 below.

report(i) Reports information to the user, including the value of the optimization algorithm

5.3 Convergence criteria

The base `OptimizationController` class provides two basic convergence criteria, that are controlled by the properties `gradtol` and `etol`.

1. The first criterion is to examine the norm of the gradient of the objective function,

$$|g_k| < \text{gradtol}$$

where `gradtol` is 1×10^{-6} by default.

2. The second is to monitor the change in the value of the objective function in successive iterations of the algorithm, f_k and f_{k+1} and stop if,

$$\begin{cases} |f_{k+1} - f_k| < \text{etol}, & |f_{k+1}| < \text{etol} \\ |f_{k+1} - f_k| < \text{etol} |f_{k+1}|, & \text{otherwise} \end{cases}$$

i.e. ensuring a relative tolerance of `etol` unless f itself is nearly zero, in which case `etol` is an absolute tolerance. The value of `etol` is 1×10^{-8} by default.

The method `hasConverged` returns **true** if either of these convergence checks are met.

5.4 Gradient descent methods

Gradient descent methods select a search direction $\mathbf{d}_k$ and update it according to the rule,

$$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \gamma_k \mathbf{d}_k$$

where γ_k is the *stepsize*.

5.4 GradientDescentController

The simplest choice, called *steepest descent*, is implemented by the `GradientDescentController` class. It simply uses the negative of the gradient of f as the search direction, i.e. $\mathbf{d}_k = -\mathbf{g}_k$ and $\gamma_k = \gamma$ constant for all iterations. This method is cheap per iteration, requiring a single evaluation of the gradient, but typically converges very slowly indeed and may oscillate around the minimum in some cases. The stepsize γ can be set in the initializer, or by changing the property `stepsize`; the default value is 0.1.

```
var control = GradientDescentController(adapter, stepsize=0.1)
```

While `GradientDescentController` is rarely useful directly, it is sometimes helpful in very constrained problems (where line searches may offer little benefit) or to establish that a new `OptimizationAdapter` is indeed working correctly.

5.4 LineSearchController

An improvement in performance can be obtained by considering the one parameter subproblem,

$$\min_{\gamma_k} f(\mathbf{x}_k + \gamma_k \mathbf{d}_k) \quad (5.4.1)$$

A `LineSearchController` aims to find a stepsize γ_k at each iteration that approximately solves this subproblem. The search direction $\mathbf{d}_k$ is by default chosen to be the steepest descent direction $\mathbf{d}_k = -\mathbf{g}_k$, but could be chosen by some other method either by subclassing `LineSearchController` and replacing the `searchDirection` method, or by performing line searches one at a time by setting the `direction` property and invoking the `step` method.

Typically, it's not necessary to solve (5.4.1) very precisely. `LineSearchController` therefore implements a *backtracking line search*, starting with $\gamma_k = 1$ and then successively reducing it by a factor $\beta \in (0, 1]$ at a time (so that $\gamma_k = \beta^n$) until the *Armijo condition*,

$$f(\mathbf{x}_k + \gamma_k \mathbf{d}_k) < f(\mathbf{x}_k) + \alpha \mathbf{g}_k \cdot \mathbf{d}_k,$$

is met, where $\alpha \in (0, 1)$ is a parameter. Success indicates that the function has decreased at least proportionately to what might be predicted from the gradient, and hence this condition helps to prevent “overshooting” the minimum. Default parameters are $\alpha = 0.2$ and $\beta = 0.5$; these can be adjusted by setting the `alpha` and `beta` in the `LineSearchController`'s initializer or by setting the properties directly.

There is a limit placed on the number of backtracking steps in a property called `maxsteps`; if this is exceeded a warning `OptLnSrchrStpsz` is generated. The `LineSearchController` also checks to ensure that $\mathbf{g}_k \cdot \mathbf{d}_k \leq 0$, i.e. that the search direction is actually downward. If this is *not* the case, a warning `OptLnSrchrDrn` is raised and it is likely that there is an error somewhere, either in the gradient calculation (if at some stage you provided one) or in the algorithm that calculated the search direction.

5.4 DirectedLineSearchController and WolfeLineSearchController

`DirectedLineSearchController` performs an Armijo search along a fixed direction, using `directionalDerivative(d)` when the merit function is nondifferentiable (`L1PenaltyAdapter`). `optimize(n)` performs a single line search. `WolfeLineSearchController` implements a strong Wolfe search and may be passed to `NewtonController` as `linesearch=WolfeLineSearchController`.

5.4 ConjugateGradientController

The conjugate gradient algorithm performs line searches, but computes the search direction using information from the prior iteration. The resulting algorithm improves convergence on objective functions that have long, narrow valleys and uses gradient information only. As for gradient descent the search direction is initially,

$$\mathbf{d}_0 = -\mathbf{g}_0,$$

and then in successive iterations the direction is computed,

$$\mathbf{d}_{k+1} = -\mathbf{g}_{k+1} + \beta_k \mathbf{d}_k.$$

The scaling β_k may be computed in several different ways, leading to different flavors of the algorithm. For example, the Fletcher-Reeves formula is,

$$\beta_k = \frac{\mathbf{g}_{k+1} \cdot \mathbf{g}_{k+1}}{\mathbf{g}_k \cdot \mathbf{g}_k}.$$

5.5 Newton and Quasi-Newton methods

Newton's method utilizes the observation that around a given point $\mathbf{x}_k$, a sufficiently smooth function can be approximated by a Taylor expansion,

$$f(\mathbf{x}) \approx f(\mathbf{x}_k) + \mathbf{g} \cdot (\mathbf{x} - \mathbf{x}_k) + \frac{1}{2}(\mathbf{x} - \mathbf{x}_k)^T \cdot \mathbf{H} \cdot (\mathbf{x} - \mathbf{x}_k) + \dots \quad (5.5.1)$$

Taking the gradient of (5.5.1) yields,

$$\nabla f(\mathbf{x}) = \mathbf{g} \cdot (\mathbf{x} - \mathbf{x}_k) + \mathbf{H} \cdot (\mathbf{x} - \mathbf{x}_k) + \dots$$

At the minimum of f , its gradient should be zero, i.e. that,

$$0 = \mathbf{g} \cdot (\mathbf{x} - \mathbf{x}_k) + \mathbf{H} \cdot (\mathbf{x} - \mathbf{x}_k) + \dots$$

and hence, neglecting higher order terms, we can obtain an update $(\mathbf{x} - \mathbf{x}_k)$ by solving the linear system,

$$\mathbf{H}_k \cdot (\mathbf{x} - \mathbf{x}_k) = -\mathbf{g}_k, \quad (5.5.2)$$

which is called a Newton step. If $f(\mathbf{x})$ is a quadratic function, then Eq. 5.5.1 is no longer an approximation and the correct solution can be obtained in one iteration. In practice, the solution of Eq. 5.5.2 is used as a search direction $\mathbf{d}_k$ for a line search.

5.5 NewtonController

The NewtonController class implements Newton updates: in each iteration a search direction is identified by solving Eq. 5.5.2 and then a line search is performed. NewtonController requires an OptimizationAdapter that can calculate a hessian and raises the error OptNoHess if it detects this is not the case.

5.5 BFGSController

The BFGS algorithm aims to compute an improved estimate of the Hessian at each iteration using only the change in parameter values $\mathbf{s}_k = \mathbf{x}_{k+1} - \mathbf{x}_k$ and the change in gradient $\mathbf{y}_k = \mathbf{g}_{k+1} - \mathbf{g}_k$. The estimate is initially set to the identity matrix,

$$\mathcal{H}_0 = \mathbf{I},$$

and then updated after each iteration using the formula,

$$\mathcal{H}_{k+1} = \mathcal{H}_k + \frac{\mathbf{y}_k \mathbf{y}_k^T}{\mathbf{y}_k^T \mathbf{s}_k} - \frac{\mathcal{H}_k \mathbf{s}_k \mathbf{s}_k^T \mathcal{H}_k^T}{\mathbf{s}_k^T \mathcal{H}_k \mathbf{s}_k}.$$

At each iteration, the current BFGS estimate of the Hessian is used to solve the linear system,

$$\mathcal{H}_k \mathbf{d}_k = -\mathbf{g}_k, \quad (5.5.3)$$

to obtain a search direction $\mathbf{d}_k$ that is then used as the basis of a linesearch.

5.5 InvBFGSController

This controller implements a variant of the BFGS algorithm that avoids the need to explicitly solve (5.5.3); instead successive estimates of the inverse Hessian, $\mathcal{H}_k^{-1}$, are built from an initial identity matrix.

5.5 LBFGSController

The LBFGS algorithm avoids the need to store an estimated inverse Hessian explicitly, instead computing its action from a stored set of $\mathbf{s}_k$ and $\mathbf{y}_k$.

5.6 Constrained optimization

5.6 PenaltyController

Implements a penalty method. Initialize with a constrained problem, and (optionally) an initial value of the penalty parameter `mu0` and a multiplier `mumul`.

```
var control = PenaltyController(adapter, mu0=1, mumul=10)
```

Internally, `PenaltyController` uses a `PenaltyAdapter` to convert the constrained problem into an unconstrained form. Successive iterations increase the penalty parameter until the constraint tolerance `ctol` is met. To perform optimization for each value of μ , `PenaltyController` uses a nested controller which is by default an `LBFGSController`, but you can choose to use an alternative by providing the `PenaltyController` constructor with the appropriate class as an optional argument `controller`.

5.6 ProjectedGradientDescentController

Projected gradient descent (`ProjectedGradientDescentController`) targets constrained problems by:

1. Removing the gradient component along *active* constraint normals (`activeConstraintGradient()`).
2. Line searching with an `L1PenaltyAdapter` merit function (`mu` defaults to 2).
3. Reprojecting onto the feasible set by minimizing $\|\mathbf{c}\|^2$ with L-BFGS (`maxconstraintsteps` iterations).

```
var opt = ProjectedGradientDescentController(adapter, mu=2, steplimit=nil)
```

Reprojection runs at `start()` and after each `step()`.

5.6 SQPController

Sequential Quadratic Programming (SQPController) is the primary constrained solver for mesh and field problems. It maintains Lagrange multipliers via an internal `LagrangeMultiplierAdapter` (`control.ladapter` after `start()`), approximates the objective Hessian with L-BFGS on the unconstrained objective (`DeconstrainAdapter`), and each iteration:

1. Builds the active constraint set from `constraintValueForActiveSet()`.
2. Solves the KKT system for the active constraints using a Schur complement of the quasi-Newton Hessian $\mathcal{H}$ (fallback to an L-BFGS step on $\nabla_{\mathbf{x}}\mathcal{L}$ if singular).
3. Line searches along the primal direction with an adaptive L1 merit function.
4. Reprojects when $\|\mathbf{c}\|_1$ exceeds `feasol` (default 10^{-5}).

```
var opt = SQPController(adapter, penalty=1, maxconstraintsteps=100,
    reprojecttol=1e-5, relgradtol=0.005,
    relgraditers=20, stagtol=1e-2, lamcap=100)
opt.optimize(500)print opt.ladapter.lagrangeMultipliers()
```

Key options include `feasol` / `reprojecttol` (feasibility tolerance), `relgradtol` and `relgraditers` (relative KKT gradient stopping), `stagtol` (objective stagnation), and `lamcap` (multiplier clamp after least-squares sync). Diagnostic methods `kkt()` and `reproject()` are available for small problems and debugging.

5.7 Custom functionals

When writing functionals for `ProblemAdapter`, provide at minimum `total(mesh)` and `integrand(mesh)` for constraints. For mesh optimization supply `gradient(mesh)`; for field optimization supply `fieldgradient(field, mesh)` and set a `field` property on the functional. Set `grade` on the functional or ensure the `integrand` width matches a mesh grade so the adapter can infer it.